

adr-0002 §4.3 step-2 device-side tuf client decision spike

field	value
Revision	1
Created	2026-06-08
Last modified	2026-06-08
Type	decision memo (spike output for adr-0002 §4.3 step-2)
Status	Recommendation: Option A (gomobile-wrapped go-tuf/v2) — proposed, pending operator review + the build-measurement gate in §9 Resolves the adr-0002 §8 item 1 / device_tuf.md §11 item 1 OPEN question: pick (A) gomobile-wrapped go-tuf/v2 vs (B) a hand-rolled Kotlin TUF client on the Security-KMP brick. Recommends A primarily on security-code-ownership grounds: A reuses the same audited go-tuf/v2 verifier the server runs (adr-0002 §3.2 / §4.2 one-trust-implementation seam), so the device does not hand-write a second security-sensitive verifier of the root→timestamp→snapshot→targets flow. The dominant cost moves from “owning crypto-correctness” (B) to “owning a JNI build seam + APK binary-size” (A), which is a measurement/engineering risk rather than a correctness risk. The recommendation is not unconditional: it is gated on the build-measurement spike in §9 (APK size delta + gomobile/JNI viability on Android-15/RK3588) and on go-tuf/v2’s metadata/updater client flow, whose API is now VERIFIED (§3) where adr-0002 §8 item 2 carried it UNVERIFIED.
Status summary	
Issues	The APK binary-size magnitude added by the gomobile Go-runtime .so is UNVERIFIED — no authoritative size

field	value
	figure was found in research; it MUST be measured (§9). gomobile/JNI viability on the specific Android-15 / RK3588 target is UNVERIFIED (adr-0002 §8 item 8). Security-KMP fit for TUF role/threshold/expiry primitives (which Option B depends on) is UNVERIFIED (adr-0002 §8 item 9) — and is the reason B is not recommended, not a reason it is. No maintained device-side Kotlin/JVM TUF <i>client</i> exists (VERIFIED by survey, §4); the nearest JVM artifact (uptane/ota-tuf) is server-side Scala.
Issues summary	Recommendation is A; the residual risk on A is build-size + JNI viability (measurable, §9), not verification correctness; B's blocker (hand-owned crypto + unverified brick fit) is the higher, less-bounded risk.
Fixed	Rev 1: enumerated the A-vs-B comparison across the six adr-0002 axes (APK size, JNI/gomobile build complexity, maintenance burden, security-code-ownership risk, brick fit, audited-verifier reuse); recommended A with rationale; fixed the integration points into the §6 verify-before-apply gate + ota-android-agent; verified the go-tuf/v2 updater/config client API (closing adr-0002 §8 item 2); verified the absence of a maintained Kotlin/JVM TUF client; carried every still-open figure as UNVERIFIED.
Fixed summary	Captured the device-client A-vs-B decision as a recommendation grounded in verified library facts, with the one genuinely unmeasured cost (APK size) carried UNVERIFIED behind a measurement gate rather than invented.
Continuation	Execute §9 step-1 (gomobile bind of a minimal go-tuf/v2 updater wrapper → measure the per-arm64 .so size delta in a release APK) and step-2 (run one full root→timestamp→snapshot→targets refresh through the wrapped updater on an Android-15 / RK3588 board) before treating A as decided; if the measured

field	value
	size delta is unacceptable AND Security-KMP is confirmed (adr-0002 §8 item 9) to expose threshold/expiry primitives, re-open B; define the offline key-custody ceremony (adr-0002 §4.3 step-3) regardless of A/B; do not mandate device-side TUF enforcement until adr-0002 §4.3 step-4.

table of contents

- [§1. purpose and scope](#)
- [§2. what the spike must decide \(the question, restated\)](#)
- [§3. verified facts — go-tuf/v2 client flow \(closes adr-0002 §8 item 2\)](#)
- [§4. verified facts — the kotlin/jvm tuf client landscape](#)
- [§5. option a vs option b — comparison](#)
- [§6. recommendation](#)
- [§7. integration points](#)
- [§8. residual risks](#)
- [§9. the measurement gate \(close before treating a as decided\)](#)
- [§10. open / unverified items carried forward](#)
- [§11. sources](#)
- [## sources verified 2026-06-08](#)

§1. purpose and scope

This is the decision memo for **adr-0002 §4.3 step-2**: the on-device TUF client spike that `device_tuf.md` §5 / §11 item 1 and adr-0002 §8 item 1 carry as the dominant, OPEN, UNVERIFIED adoption cost. It picks between the two options those documents enumerate but deliberately did **not** decide (per HelixConstitution §11.4.6 no-guessing / §11.4.8 research-before-implementation):

- **Option A** — gomobile-wrapped go-tuf/v2 (compile the Go metadata/updater client flow to an Android library via gomobile/JNI; the Kotlin ota-android-agent calls the wrapped Go).
- **Option B** — a hand-rolled Kotlin TUF client implementing the root→timestamp→snapshot→targets refresh/verify flow natively against the Security-KMP brick's crypto primitives.

In scope: the verified library + landscape facts the choice rests on (§3, §4); the A-vs-B comparison across the six adr-0002 axes (§5); a concrete recommendation + rationale (§6); where the chosen client plugs into the existing verify-before-apply gate and the ota-android-agent (§7); the residual risks (§8); and the measurement gate that must close before A is treated as decided (§9).

Out of scope / not re-decided here. TUF adoption itself (adr-0002, decided phased). The server-side publish path (go-tuf/v2, adr-0002 §3.2 / §4.3 step-1 — DECIDED). The device verification *flow and invariants* (`device_tuf.md` §6, fixed there; this memo decides only *what code* runs that flow). The offline key-custody

ceremony (adr-0002 §4.3 step-3; `key_management.md`). The Director/Image stretch split (adr-0002 §4.3 step-5). The TUF metadata storage DDL (`device_tuf.md` §9). Making device-side enforcement *mandatory* (adr-0002 §4.3 step-4 — happens only after this spike + the ceremony close their UNVERIFIED items).

LOCKED context honored (not re-decided): native Android A/B (update_engine + AVB/dm-verity + auto-rollback) + a custom Go (Gin) control plane; Go + go-tuf/v2 server-side; Android-15-first (Orange Pi 5 Max / RK3588); the Kotlin agent (ota-android-agent) is the device-side consumer (adr-0002 §1, §4; `device_tuf.md` §1).

§2. what the spike must decide (the question, restated)

`device_tuf.md` §6 already fixes the device-side **flow and invariants** the client must enforce — role order root→timestamp→snapshot→targets, threshold-signature verification per the current root, expiry rejection, per-role monotonic-version rejection (rollback/freeze guard), snapshot[?]targets consistency (mix-and-match guard), and the TUF-targets-digest [?] recomputed-SHA-256 cross-binding ahead of the unchanged MVP hash+signature check. The flow is **not** what this spike decides.

What this spike decides is **which implementation runs that flow on the device** — i.e. who owns the most security-sensitive code in the whole OTA chain: the metadata verification logic. That is the framing adr-0002 §5.2 gives the two options: A's cost is "Go toolchain on the Android build + APK binary-size + a JNI boundary"; B's cost is "owning security-sensitive verification code ... the highest-risk thing to hand-write." The decision axis is therefore **risk class**, not feature parity (both options enforce the identical §6 flow).

§3. verified facts — go-tuf/v2 client flow (closes adr-0002 §8 item 2)

adr-0002 §8 item 2 and `device_tuf.md` §11 item 2 carried the go-tuf/v2 metadata/updater / metadata/trustedmetadata client-flow API as **UNVERIFIED** (pkg.go.dev was truncated when the ADR was written). Verified against pkg.go.dev on **2026-06-08**:

Module / version. `github.com/theupdateframework/go-tuf/v2`; latest release **v2.4.2 (published 2026-05-19)** — consistent with the v2.4.2 / spec-1.0.31 fact adr-0002 §3.2 recorded for 2026-06-07.

metadata/updater package — VERIFIED exported surface (signatures quoted from pkg.go.dev):

```
func New(config *config.UpdaterConfig) (*Updater, error)
```

```
func (update *Updater) Refresh() error
```

```
func (update *Updater) GetTargetInfo(targetPath string)  
    (*metadata.TargetFiles, error)
```

```

func (update *Updater) FindCachedTarget(targetFile
    *metadata.TargetFiles, filePath string) (string, []byte,
    error)
func (update *Updater) DownloadTarget(targetFile
    *metadata.TargetFiles, filePath, targetBaseURL string)
    (string, []byte, error)
func (update *Updater) GetTopLevelTargets()
    map[string]*metadata.TargetFiles
func (update *Updater) GetTrustedMetadataSet()
    trustedmetadata.TrustedMetadata

```

Refresh() runs the full root→timestamp→snapshot→targets refresh+verify workflow against locally cached + remotely fetched metadata; GetTargetInfo(path) resolves the requested artifact in targets and returns its authorized *metadata.TargetFiles (path + length + hashes) — exactly the device_tuf.md §6.1 “resolve the requested artifact in targets; capture its authorized length + SHA-256” step. FindCachedTarget / DownloadTarget enforce the declared-length + hash binding on the bytes.

metadata/config package — VERIFIED UpdaterConfig (signatures quoted from pkg.go.dev):

```

func New(remoteURL string, rootBytes []byte) (*UpdaterConfig,
    error)    // takes the pinned bootstrap root.json bytes

type UpdaterConfig struct {
    MaxRootRotations    int64    // root-rotation bound – the §7.2
        chained root→root walk cap
    MaxDelegations      int
    RootMaxLength        int64
    TimestampMaxLength   int64
    SnapshotMaxLength    int64
    TargetsMaxLength     int64    // endless-data cap (device_tuf.md
        §4)

    Fetcher              fetcher.Fetcher    // pluggable HTTP
        fetcher → the existing HTTP/3→HTTP/2 path
    LocalTrustedRoot      []byte    // the pinned, image-
        embedded root.json (device_tuf.md §7.1)
    LocalMetadataDir      string    // on-device trusted-
        metadata cache (device_tuf.md §6.3)
    LocalTargetsDir       string
    RemoteMetadataURL     string
    RemoteTargetsURL      string
    DisableLocalCache     bool
    PrefixTargetsWithHash bool    // consistent-snapshot layout toggle
    UnsafeLocalMode       bool
}
// SetDefaultFetcherHTTPClient(*http.Client),
// SetDefaultFetcherTransport(http.RoundTripper),
// SetDefaultFetcherRetry(...), EnsurePathsExist() also exported.

```

Why this matters for the decision. The verified surface shows the client flow is a small, well-shaped API: construct UpdaterConfig from the **pinned bootstrap root.json bytes** (LocalTrustedRoot — the device_tuf.md §7.1 trust-on-first-install anchor), call Refresh(), call GetTargetInfo(). A gomobile wrapper (Option A) needs to surface ~3 calls across JNI (New → Refresh → GetTargetInfo, returning length+SHA-256). Conversely, Option B would have to re-implement everything Refresh() does — root rotation bounds, four expiry checks, four monotonic-version checks, threshold counting, snapshot[?]targets consistency — in hand-written Kotlin. The verified API makes A’s wrapping surface **small and known**, while leaving B’s re-implementation surface **large and security-critical**.

§4. verified facts — the kotlin/jvm tuf client landscape

adr-0002 §8 item 1 / device_tuf.md §5 stated “no verified production-grade Kotlin/JVM TUF client is known to exist.” Re-surveyed on **2026-06-08** (theupdateframework.io, the TUF GitHub org, and a targeted JVM/Maven search):

- **No official TUF Java/Kotlin implementation** exists in the theupdateframework.org. The maintained official implementations are Python (reference), Go (go-tuf), JavaScript (tuf-js), and Rust (rust-tuf). (theupdateframework GitHub org / TUF site, accessed 2026-06-08.)
- **uptane/ota-tuf** (the nearest JVM artifact) is **Scala**, and — VERIFIED from its README — is a **server-side** TUF repository + key-management service (reposerver for metadata administration, keyserver for online role signing, a CLI, libtuf/libtuf-server). It is **not** a device-side verification client. Actively maintained (latest release v6.1.0, 2026-04-27) but the wrong tier: it publishes/signs metadata, it does not verify metadata on an embedded device. (github.com/uptane/ota-tuf, accessed 2026-06-08.)
- No actively-maintained, Maven-published **JVM TUF client** (the device-verification role) was found.

Conclusion (VERIFIED-as-absence): the third path adr-0002 §3.2 considered — “an existing JVM client” — still has **no maintained candidate**. This removes the cheapest hypothetical option and sharpens the choice to A vs B exactly as the ADR framed it. Critically, it also means Option B is **not** “wrap a maintained JVM library”; it is “**write a new TUF client from scratch in Kotlin**” — the highest-risk reading of B.

§5. option a vs option b — comparison

Across the six axes the task and adr-0002 §3.2 / §5.2 name. “VERIFIED” / “UNVERIFIED” tags per HelixConstitution §11.4.6.

axis	option A — gomobile-wrapped go-tuf/v2	option B — hand-rolled Kotlin on Security-KMP
APK / binary size	Adds the Go runtime .so per ABI to the APK.	Adds only Kotlin/JVM bytecode + whatever

axis	option A — gomobile-wrapped go-tuf/v2	option B — hand-rolled Kotlin on Security-KMP
	Magnitude UNVERIFIED — no authoritative size figure was found; the Go-runtime native lib is a well-known non-trivial multi-MB-per-arch cost, but the exact bytes MUST be measured (§9). Mitigable: ship arm64-v8a only for the RK3588 target (-target=android/arm64), which avoids the fat-APK multi-ABI multiplier. Adds the Go toolchain + gomobile + NDK to the Android build and a JNI boundary. gomobile is maintained (recent x/mobile + NDK 28 / 16KB-page-size support, 2025); the wrapped surface is small (§3, ~3 calls). Viability on Android-15 / RK3588 is UNVERIFIED (adr-0002 §8 item 8) and is the §9 gate.	Security-KMP already pulls in. Smallest footprint; no native runtime. VERIFIED-directional (no Go runtime), exact delta still board-dependent.
JNI / gomobile build complexity		No Go toolchain, no JNI, no NDK coupling — pure Kotlin in the existing agent build. Lowest build-system complexity. VERIFIED-directional.
maintenance burden	Track one verifier: bump go-tuf/v2 (Apache-2.0, actively maintained, latest v2.4.2 2026-05-19, §3) and re-bind. Server and device share the same library + the same spec version, so a TUF-spec change lands once.	Track a hand-written security codebase forever: every TUF-spec change, every CVE class, every attack-class refinement must be re-derived and re-tested in Kotlin independently of the server. Divergence between the Kotlin verifier and the Go server verifier is a standing risk.
security-code-ownership risk	LOW — reuses the same audited go-tuf/v2 verification logic the server runs (adr-0002 §3.2 / §4.2 “one trust implementation, no second hand-written verifier”). The most	HIGH — means owning security-sensitive verification code, “<i>the highest-risk thing to hand-write</i>” (adr-0002 §5.2). §4 shows there is no maintained JVM client to start from, so B

axis	option A — gomobile-wrapped go-tuf/v2	option B — hand-rolled Kotlin on Security-KMP
	security-sensitive code (metadata verification) is not newly authored.	is a from-scratch TUF client. A subtle hand-rolled error (threshold off-by-one, missed expiry, non-monotonic accept) is exactly the attack surface TUF exists to close.
Security-KMP brick fit	Not on the critical path — A does its crypto inside go-tuf/v2; Security-KMP is not required for verification. (Security-KMP may still host the MVP detached-sig device re-check, unchanged.)	On the critical path and UNVERIFIED — B depends on Security-KMP exposing (or being extended to host, without bespoke crypto) TUF role/threshold/expiry primitives. adr-0002 §8 item 9 / device_tuf.md §11 item 3 carry this fit as UNVERIFIED . If the brick lacks threshold/multi-key primitives, B either grows bespoke crypto (forbidden direction per §11.4.74 catalogue-first) or stalls.
audited-verifier reuse	YES — the defining advantage. One trust implementation, server + device, byte-for-byte the same role-verification semantics.	NO — a second, independent verifier the project must prove equivalent to the Go one.

Symmetry note (no bluff). A is **not** risk-free: it trades a *correctness-ownership* risk (B) for a *build/size + JNI-viability* risk (A). The decisive difference is that A's residual risk is **bounded and measurable** (you can build the wrapper and read the APK size; you can run one refresh on the board), whereas B's residual risk is **unbounded and ongoing** (you can never fully prove a hand-written security verifier is free of subtle attack-class gaps, and you must re-prove it on every spec change).

§6. recommendation

Recommend Option A — gomobile-wrapped go-tuf/v2 — conditional on the §9 measurement gate.

Top three rationale points:

1. **Security-code-ownership: reuse the audited verifier, don't hand-write a second one.** The metadata verification flow is the single most security-sensitive component of the OTA trust chain (its whole purpose is to be un-foolable by a

repository/key compromise — adr-0002 §2). A runs the **same audited go-tuf/v2 Refresh() verifier the server uses** (§3; adr-0002 §4.2 one-trust-implementation seam), so the device adds **zero** new hand-written security-critical code. B requires authoring a from-scratch Kotlin TUF client (§4 confirms there is no maintained JVM client to lean on), which adr-0002 §5.2 calls “*the highest-risk thing to hand-write.*” Risk-class beats footprint for a safety-critical path (HelixConstitution §11.4.123 rock-solid-proof — a hand-rolled verifier is the hardest thing to prove correct).

2. **A’s cost is bounded + measurable; B’s cost is unbounded + ongoing.** A’s only genuine unknown is APK binary-size + JNI viability on RK3588 — both *measurable in a single spike* (§9) and mitigable (arm64-only build avoids the fat-APK multiplier; gomobile is maintained with current NDK-28 / 16KB-page-size support). B’s cost is permanent: re-deriving + re-testing TUF correctness in Kotlin on every spec change, plus the **UNVERIFIED** Security-KMP threshold/expiry fit (adr-0002 §8 item 9) that B is *on the critical path for* and A is not.
3. **The verified API makes A small and B large.** §3 shows the client flow is ~3 wrapped calls (`config.New(remoteURL, rootBytes) → updater.New → Refresh() → GetTargetInfo()`), mapping 1:1 onto the `device_tuf.md` §6 flow (`pinned root.json → LocalTrustedRoot; resolve target → GetTargetInfo`). A wraps a tiny, known surface; B must re-implement everything behind `Refresh()` (root-rotation bounds, four expiry checks, four monotonic-version checks, threshold counting, snapshot[?]targets consistency).

This recommendation flips to B only if the §9 measurement shows the arm64-only Go-runtime .so size delta is unacceptable for the product **and** adr-0002 §8 item 9 closes with Security-KMP confirmed to expose the TUF threshold/expiry primitives B needs. Absent both, A is the lower-total-risk choice. Per adr-0002 §4.3 step-4, device-side TUF enforcement is **not made mandatory** until this spike + the key-custody ceremony close their UNVERIFIED items either way.

§7. integration points

The chosen client (A) plugs into two existing, unchanged seams. It does **not** alter the `device_tuf.md` §6 flow/invariants or the MVP gate.

§7.1 the verify-before-apply gate (`device_tuf.md` §6 / `signing_verification.md` §6, §8 seam 3)

The TUF refresh/verify runs as the **new first gate, in front of** the unchanged MVP hash+signature check (`device_tuf.md` §6.1, §8.2 five-gate chain). With Option A the wrapped go-tuf/v2 updater fills exactly the [NEW] TUF metadata refresh + verify box:

download byte-identical artifact (HTTPS+Range; ADR-0004)



```
[wrapped go-tuf/v2 updater]  cfg = config.New(remoteURL,
pinnedRootBytes)           // §7.1 image-embedded root.json →
LocalTrustedRoot
```

u =

```

updater.New(cfg)                                // §3

u.Refresh()                                     //
root→timestamp→snapshot→targets; expiry + monotonic + threshold +
consistency

                                ti =
u.GetTargetInfo(targetPath)                    // authorized length +
SHA-256 for THIS artifact
    |   ANY error from Refresh()/GetTargetInfo → ABORT before
    |   apply (device_tuf.md §6.1; do NOT fall back to MVP-only)
    ▼
MVP gate (UNCHANGED): recompute SHA-256/512 → MUST equal ti.Hashes
(TUF-digest ↔ recomputed-bytes
    cross-binding, device_tuf.md §6.2) AND the manifest hash →
verify ED25519 detached sig vs trust-store key_id
    ▼
hand fully-verified local artifact to ota-update-engine-bridge →
applyPayload
    ▼
update_engine FILE_HASH/METADATA_HASH + payload sig (native gate
3) → AVB / A/B auto-rollback (native gate 4)

```

The cross-binding in device_tuf.md §6.2 is satisfied directly by comparing the agent's recomputed SHA-256 to ti.Hashes from GetTargetInfo — the verified API returns exactly that.

§7.2 the ota-android-agent (README §7; device_tuf.md §5)

- **Wrapped library packaging.** gomobile bind -target=android/arm64 ./tufclient produces an .aar the Kotlin ota-android-agent depends on; the agent calls a thin Kotlin facade (TufClient.refreshAndResolve(targetPath): {length, sha256}) over the JNI boundary. Build the arm64-only variant for the RK3588 target to avoid the fat-APK multiplier (§5).
- **Trusted-metadata cache + bootstrap root.** UpdaterConfig.LocalMetadataDir points at an app-private on-device cache (the device_tuf.md §6.3 on-disk trusted-metadata layout); UpdaterConfig.LocalTrustedRoot is loaded from the **image-embedded, AVB-protected pinned root.json** (device_tuf.md §7.1; the exact on-image location is the UNVERIFIED board item, adr-0002 §8 item 8).
- **Fetcher.** UpdaterConfig.Fetcher (or SetDefaultFetcherHttpClient) routes metadata + target fetches over the existing HTTP/3→HTTP/2 transport (ADR-0004); no separate network stack.
- **Refresh trigger.** The refresh fires at the update-check poll seam (rollout_engine.md §4 deterministic cohort check) ahead of the per-device install decision (the future Director seam, device_tuf.md §5). The exact trigger relative to the poll + the failure→agent-state mapping are the §6.3 fields the spike finalizes.
- **Security-KMP** remains the home of the **unchanged MVP detached-signature device re-check** (it is not on the TUF-verification path under A). No bespoke crypto is introduced (§11.4.74).

§8. residual risks

1. **APK binary-size (A).** Go-runtime `.so` per ABI. **UNVERIFIED** magnitude — measure (§9). Mitigation: arm64-only build for RK3588; if still unacceptable, reconsider B (§6 flip condition).
2. **gomobile/JNI viability on Android-15 / RK3588 (A).** **UNVERIFIED** (adr-0002 §8 item 8). Mitigation: §9 step-2 runs one full refresh on the board before A is treated as decided.
3. **JNI error-surface mapping.** Errors from `Refresh()/GetTargetInfo()` must map cleanly across JNI into the agent's abort-before-apply path (no silent swallow). Covered by the `device_tuf.md` §10 Layer-4 mutation “skip the TUF refresh / force verify always-true” mutants — they must flip PASS→FAIL through the wrapper.
4. **gomobile project health (A).** A long-lived dependency on `golang.org/x/mobile`. Maintained as of 2025 (NDK-28 / 16KB-page-size support) but a single-point maintenance dependency. Mitigation: the wrapped surface is tiny (§3), so a future re-wrap or fallback is bounded.
5. **Security-KMP TUF-primitive fit (B-side, carried).** Still **UNVERIFIED** (adr-0002 §8 item 9). Only bites if A is rejected at §9 and B is reopened; it is the reason B is not the default.
6. **Root-of-trust bootstrap provenance (both).** The pinned `root.json` must be image-embedded + AVB-covered (`device_tuf.md` §7.1); its exact on-image location is **UNVERIFIED** board work (adr-0002 §8 item 8), independent of A/B.
7. **No production precedent (both).** No shipping production Android+TUF device client is known (§4; adr-0002 §8 item 6) — A is the first, which is itself an argument for reusing the audited verifier rather than hand-writing one.

§9. the measurement gate (close before treating a as decided)

A is recommended **conditionally**; these two measurements close the only UNVERIFIED risks unique to A (HelixConstitution §11.4.123 — decide on measured evidence, not assertion):

- **Step 1 — APK size delta.** `gomobile bind -target=android/arm64` a minimal wrapper exposing `config.New` → `updater.New` → `Refresh` → `GetTargetInfo`; integrate the `.aar` into a release build of `ota-android-agent`; **measure the per-arm64 .so size and the net signed-APK delta.** Record the number (it is currently UNVERIFIED). Decision rule: accept A if the delta is within the product's size budget; otherwise evaluate the §6 flip condition.
- **Step 2 — JNI viability on the board.** Run **one full** `root`→`timestamp`→`snapshot`→`targets` `Refresh()` + `GetTargetInfo()` through the wrapped updater on an **Android-15 / RK3588 (Orange Pi 5 Max)** device against the §4.3 step-1 server publish path, with a pinned bootstrap `root.json`. Capture positive evidence (the resolved length+SHA-256 matching a known target) and run the `device_tuf.md` §10 Layer-3 attack-class rejections (rollback/freeze/mix-and-match) through the wrapper. A is decided only when both steps pass.

§10. open / unverified items carried forward

1. **APK binary-size magnitude for the gomobile Go-runtime .so** — **UNVERIFIED**; no authoritative figure found; measure in §9 step-1. (Not invented per §11.4.6.)
2. **gomobile/JNI viability on Android-15 / RK3588** — **UNVERIFIED** (adr-0002 §8 item 8); §9 step-2.
3. **Security-KMP fit for TUF role/threshold/expiry primitives** — **UNVERIFIED** (adr-0002 §8 item 9); only material if A is rejected and B reopened.
4. **On-image AVB-protected location of the pinned root.json** — **UNVERIFIED** board work (device_tuf.md §7.1; adr-0002 §8 item 8).
5. **§6.3 client fields** — on-disk cache layout, refresh trigger vs poll, failure→agent-state mapping — finalized by this spike's implementation; flow/invariants already fixed in device_tuf.md §6.
6. **HelixConstitution clause numbers** (§11.4.6, §11.4.8, §11.4.74, §11.4.123) cited per corpus convention; the Constitution source file is not present in this repo (adr-0002 §7 note). **UNVERIFIED**.
7. **No shipping production Android+TUF device client precedent** — **UNVERIFIED/absent** (§4; adr-0002 §8 item 6).

§11. sources

In-repo (paths relative to docs/research/main_specs/):

- [research/adr/adr-0002-supply-chain-trust.md](#) — §3.2 (go-tuf/v2 verified facts + additive layering), §4.2 (one-trust-implementation seam, verify-gate-distinct, Security-KMP routing), §4.3 (step-1 server+Go-client prototype, step-2 this spike, step-3 ceremony, step-4 mandatory-gating, step-5 Director split), §5.2 (A vs B cost framing), §8 (UNVERIFIED items 1, 2, 6, 8, 9).
- [1.0.1-staged-rollout/device_tuf.md](#) — §3 (roles + key posture), §4 (attack classes), §5 (server↔device split, the A/B option table), §6 (device verify-before-apply flow + §6.2 cross-binding + §6.3 OPEN fields), §7.1 (pinned bootstrap root.json), §10 (four-layer + attack-class testing), §11 (OPEN items).
- [1.0.0-mvp/security/signing_verification.md](#) — §4 (build-pipeline key in device trust store, same provenance as the pinned root.json), §6 (device re-verify gate this client fronts), §8 (TUF-forward seams).
- [1.0.1-staged-rollout/rollout_engine.md](#) — §4 (cohort check = refresh-trigger seam).

External (URLs + access date in the footer):

- go-tuf/v2 metadata/updater package — pkg.go.dev (v2.4.2, 2026-05-19); the Updater API (New, Refresh, GetTargetInfo, FindCachedTarget, DownloadTarget, GetTopLevelTargets).
- go-tuf/v2 metadata/config package — pkg.go.dev; UpdaterConfig (New(remoteURL, rootBytes), LocalTrustedRoot, Fetcher, *MaxLength, MaxRootRotations, PrefixTargetsWithHash).
- The Update Framework project + GitHub org — official implementations (Python/Go/JS/Rust); no official Java/Kotlin client.
- uptane/ota-tuf GitHub README — Scala, **server-side** reposerver/keyserver (latest v6.1.0, 2026-04-27); not a device client.

- gomobile / golang.org/x/mobile — gomobile bind, -target=android/arm64 ABI selection; maintained 2025 (NDK-28, 16KB page-size alignment); per-ABI Go-runtime .so (exact size UNVERIFIED).

sources verified 2026-06-08

- go-tuf/v2 metadata/updater — <https://pkg.go.dev/github.com/theupdateframework/go-tuf/v2/metadata/updater> — accessed 2026-06-08 (latest release v2.4.2, 2026-05-19; Updater/New/Refresh/GetTargetInfo signatures verified).
- go-tuf/v2 metadata/config — <https://pkg.go.dev/github.com/theupdateframework/go-tuf/v2/metadata/config> — accessed 2026-06-08 (UpdaterConfig.New(remoteURL, rootBytes) + field set verified).
- go-tuf repository — <https://github.com/theupdateframework/go-tuf> — accessed 2026-06-08 (v2 replaces legacy v0.7.0; Updater/TrustedMetadata/Refresh workflow description).
- The Update Framework — <https://theupdateframework.io/> and <https://github.com/theupdateframework> — accessed 2026-06-08 (official implementations are Python/Go/JS/Rust; no official Java/Kotlin client).
- uptane/ota-tuf — <https://github.com/uptane/ota-tuf> — accessed 2026-06-08 (Scala, server-side reposerver+keyserver, latest v6.1.0 2026-04-27; NOT a device verification client).
- gomobile — <https://pkg.go.dev/golang.org/x/mobile/cmd/gomobile> and <https://danballard.com/2025/09/28/generating-android-16kb-page-size-libraries-from-go/> — accessed 2026-06-08 (-target ABI selection; NDK-28 / 16KB-page-size support 2025; no authoritative Go-runtime .so size figure found → carried UNVERIFIED).
- **Negative finding (no maintained JVM/Kotlin TUF device client):** targeted GitHub/Maven survey — accessed 2026-06-08 — nearest JVM artifact is Scala server-side uptane/ota-tuf; no maintained Kotlin/JVM device-verification client exists.